

QUESTION 1

TABLE - CREATE TABLE Customer (

customer_id INT PRIMARY KEY,

customer_name VARCHAR(50),

city VARCHAR(50)

);

CREATE TABLE Orders (

order_id INT PRIMARY KEY,

customer_id INT REFERENCES Customer(customer_id),

amount INT

);

INSERT INTO Customer (customer_id, customer_name, city)

VALUES

(1, 'Aman', 'Delhi'),

(2, 'Neha', 'Delhi'),

(3, 'Raj', 'Mumbai'),

(4, 'Priya', 'Mumbai'),

(5, 'Karan', 'Pune');

INSERT INTO Orders (order_id, customer_id, amount)

VALUES

(101, 1, 4000),

(102, 1, 3000),

(103, 2, 5000),

(104, 3, 6000),

(105, 3, 2000),

(106, 4, 8000),

(107, 5, 4500);

SOLUTION :-

SELECT t.customer_name, t.city, t.total_purchase

FROM (

SELECT c.customer_id, c.customer_name, c.city,

SUM(o.amount) AS total_purchase

FROM Customer c

JOIN Orders o

ON c.customer_id = o.customer_id

GROUP BY

c.customer_id, c.customer_name, c.city

) AS t

WHERE NOT EXISTS (

SELECT 1

FROM Customer c2

JOIN Orders o2

ON c2.customer_id = o2.customer_id

WHERE c2.city = t.city

GROUP BY

c2.customer_id,

c2.customer_name

HAVING SUM(o2.amount) > t.total_purchase

);

OUTPUT

	customer_name character varying (50)	city character varying (50)	total_purchase bigint
1	Karan	Pune	4500
2	Priya	Mumbai	8000
3	Aman	Delhi	7000
4	Raj	Mumbai	8000

QUESTION 2:-

```
CREATE TABLE product_inventory (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
    price INT
);
```

```
INSERT INTO product_inventory (product_id, product_name, price)
VALUES
(101, 'Laptop', 65000),
(102, 'Mobile', 30000);
```

```
CREATE TABLE product_audit (
    audit_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id INT,
```

```
product_name VARCHAR(50),  
action VARCHAR(30),  
action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE OR REPLACE FUNCTION log_product_insert()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO product_audit  
        (product_id, product_name, action)  
VALUES  
    (NEW.product_id, NEW.product_name, 'PRODUCT ADDED');  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER after_product_insert  
AFTER INSERT ON product_inventory  
FOR EACH ROW  
EXECUTE FUNCTION log_product_insert();
```

```
CREATE OR REPLACE FUNCTION log_product_delete()  
RETURNS TRIGGER AS $$  
BEGIN
```

```
INSERT INTO product_audit
    (product_id, product_name, action)
VALUES
    (OLD.product_id, OLD.product_name, 'PRODUCT REMOVED');

RETURN OLD;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER after_product_delete
AFTER DELETE ON product_inventory
FOR EACH ROW
EXECUTE FUNCTION log_product_delete();

INSERT INTO product_inventory (product_id, product_name, price)
VALUES (103, 'Tablet', 20000);

DELETE FROM product_inventory
WHERE product_id = 101;

SELECT * FROM product_audit;
```

	audit_id [PK] integer	product_id integer	product_name character varying (50)	action character varying (30)	action_time timestamp without time zone
1	1	103	Tablet	PRODUCT ADDED	2026-10-09 10:08:23.95638
2	2	101	Laptop	PRODUCT REMOVED	2026-10-09 10:08:23.95638